

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _____

Assessment Tasks Case Study 1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations. 2. **Case Study 2: Cloud-Based Document Collaboration System**

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration. 3. **Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case Study 1: Smart Retail Inventory Management System

1. Understanding of the Case

The retail chain needs a centralized, cloud-based inventory management system that connects distributed store devices (POS terminals and dashboards) to a common backend. Core business requirements are: real-time visibility of stock across stores, secure and reliable communication between clients and cloud services, centralized storage of product, transaction, and analytics data, and intelligent demand forecasting to reduce stock-outs and overstocking.

2. Application of Cloud Concepts

- **Client Tier:** POS systems and web/mobile dashboards act as clients that consume RESTful Web APIs to fetch product data and push transaction updates.
- **API Gateway:** A managed API Gateway (e.g., AWS API Gateway/Azure API Management) exposes secure, versioned endpoints, handles authentication, rate limiting, and routing to backend microservices.
- **Cloud Storage:** Structured product/transaction data is stored in a managed relational database (e.g., Amazon RDS/Azure SQL); large analytics datasets and logs are stored in object storage (e.g., Amazon S3) or a data lake for cost-effective scaling.
- **Platform Services (PaaS):** Managed analytics and ML services (e.g., AWS SageMaker/Azure ML, or a serverless analytics pipeline with AWS Glue + Athena) are used for demand forecasting without managing underlying infrastructure.
- **Security:** VPNs connect store networks to the cloud VPC; firewalls and security groups restrict traffic; the API Gateway enforces token-based authentication (OAuth2/JWT) for every request.

3. Analysis / Trade-offs

- **Trade-off — Consistency vs Availability:** Inventory counts across stores favor eventual consistency for dashboards (fast reads) but transactional writes (sales) require strong consistency to avoid overselling; a hybrid model (strongly consistent OLTP DB + eventually consistent analytics store) balances this.
- **Trade-off — Cost vs Latency:** Using a managed PaaS forecasting service reduces operational overhead but may add latency/cost per API call compared to self-hosted models; acceptable given lower DevOps burden.
- **Security vs Usability:** VPN + firewall + API gateway layering adds slight latency but is justified because financial/transactional data demands defense-in-depth.
- **Scalability:** Auto-scaling API backend and storage handle peak sales periods (festive season) without over-provisioning during low traffic.

4. Solution Design

Proposed Architecture: A 3-tier design — Client Tier (POS & dashboards) → API/Integration Tier (API Gateway + microservices) → Data/Platform Tier (cloud storage + managed analytics/ML service) — connected securely through VPN and firewall-protected channels.

- POS/Dashboard (Client) → API Gateway (Auth, Routing) → Microservices (Inventory, Transaction, Forecast) → RDS (transactional data) + S3/Data Lake (logs, analytics).

- Forecasting microservice periodically pulls data from the data lake and pushes demand predictions back via an internal API for dashboard consumption.
- VPN tunnels connect each store's local network to the cloud VPC; firewalls/security groups restrict inbound traffic to only the API Gateway endpoint.
- CI/CD pipeline and monitoring (e.g., CloudWatch) ensure high availability and quick rollback if an update fails.

5. Conclusion

The proposed architecture is feasible, scalable, and secure, effectively integrating clients, web APIs, cloud storage, and platform services to satisfy the business requirements outlined in the case, while remaining cost-efficient and maintainable.

Case Study 2: Cloud-Based Document Collaboration System

1. Understanding of the Case

The enterprise requires a real-time, multi-user document collaboration platform (similar to Google Docs) accessible from any browser. Key requirements include simultaneous multi-user editing with conflict resolution, reliable version history, global low-latency access, and strong security for sensitive business documents including encryption, identity management, and fine-grained access control.

2. Application of Cloud Concepts

- **Client Tier:** Thin browser-based clients use WebSockets/HTTP for real-time updates and RESTful Web APIs for document CRUD operations.
- **Cloud Storage:** Documents and their revisions are stored in distributed, versioned object storage (e.g., Amazon S3 with versioning, or Google Cloud Storage) while metadata (permissions, ownership) sits in a fast NoSQL store (e.g., DynamoDB/Firestore).
- **Web APIs & Sync:** A synchronization service uses Operational Transformation (OT) or Conflict-free Replicated Data Types (CRDTs) exposed via WebSocket APIs to merge concurrent edits in real time across devices.
- **Global Infrastructure:** A Content Delivery Network (CDN) and multi-region deployment with load balancers reduce latency for globally distributed users; edge caching serves static assets.
- **Security:** TLS/HTTPS encrypts data in transit; server-side encryption (SSE) protects data at rest; an Identity and Access Management (IAM) service (e.g., OAuth2/OpenID Connect, Azure AD) manages authentication and role-based access permissions (owner, editor, viewer).

3. Analysis / Trade-offs

- **Trade-off — Real-time Sync Complexity vs UX:** CRDT/OT-based sync is complex to implement but essential for a seamless, conflict-free collaborative experience; simpler last-write-wins would degrade UX.
- **Trade-off — Global Availability vs Consistency:** Multi-region replication improves latency and availability but introduces eventual consistency risk for simultaneous edits from different regions, mitigated by the CRDT merge layer.
- **Cost vs Redundancy:** Storing full version history for every document increases storage cost but is necessary for auditability and recovery; lifecycle policies can archive old versions to cheaper cold storage.
- **Security vs Performance:** End-to-end encryption maximizes confidentiality but can complicate real-time collaborative decryption; a practical compromise is encryption at rest/in transit plus strict IAM rather than full E2E encryption.

4. Solution Design

Proposed Architecture: Browser clients connect over secure WebSockets to a real-time sync layer (CRDT/OT) backed by versioned distributed cloud storage and a metadata/permissions database, fronted by a global CDN/load balancer for low-latency worldwide access.

- Browser Client (Editor UI) ↔ WebSocket Sync Service (CRDT engine) ↔ Application Servers ↔ Metadata DB (NoSQL) + Versioned Object Storage.
- Global Load Balancer + CDN route users to the nearest regional deployment; regions replicate metadata asynchronously.

- IAM service issues short-lived access tokens; every API call is authorized against document-level permissions before reaching storage.
- Automated version snapshots on each save enable rollback and audit trails.

5. Conclusion

The proposed architecture is feasible, scalable, and secure, effectively integrating clients, web APIs, cloud storage, and platform services to satisfy the business requirements outlined in the case, while remaining cost-efficient and maintainable.

Case Study 3: Online Food Ordering Platform

1. Understanding of the Case

The startup needs a scalable food ordering platform accessible via web and mobile clients, capable of handling variable traffic (peak meal-time surges), securely storing menu images, invoices, and customer data, and reliably processing orders through backend services. Requirements include RESTful integration between frontend and backend, scalable compute infrastructure, and robust authentication and authorization.

2. Application of Cloud Concepts

- **Client Tier:** Web and mobile apps (React/Flutter) communicate with backend microservices via RESTful Web APIs (JSON over HTTPS).
- **Compute/Platform:** Backend services (Order, Menu, Payment, Notification) are containerized and orchestrated using managed Kubernetes (e.g., Amazon EKS/Azure AKS) for auto-scaling and resilience; virtual machines handle any legacy or stateful workloads.
- **Cloud Storage:** Menu images are stored in object storage (e.g., Amazon S3) served via a CDN for fast image delivery; invoices and customer data are stored in a managed relational database (e.g., Amazon RDS/Cloud SQL) with encryption enabled.
- **Security:** Authentication uses JWT/OAuth2 tokens; all traffic is served over HTTPS/TLS; Role-Based Access Control (RBAC) differentiates customers, restaurant partners, delivery agents, and admins.
- **Scalability:** Kubernetes Horizontal Pod Autoscaler scales order/payment services during lunch/dinner rush; a message queue (e.g., Amazon SQS/Kafka) decouples order placement from kitchen/delivery processing.

3. Analysis / Trade-offs

- **Trade-off — Scalability vs Cost:** Managed Kubernetes auto-scaling handles traffic spikes efficiently but incurs higher baseline cost than static VMs; justified by unpredictable, spiky food-ordering traffic patterns.
- **Trade-off — Synchronous vs Asynchronous Processing:** Using a message queue for order processing improves resilience and decouples services, at the cost of slightly more complex eventual-consistency handling for order status.
- **Security vs Convenience:** Token-based auth with short expiry improves security but requires refresh-token handling to preserve a smooth mobile UX.
- **Reliability:** Multi-AZ deployment of the database and Kubernetes cluster protects against zone failures, critical for a customer-facing, revenue-generating platform.

4. Solution Design

Proposed Architecture: Web/mobile clients call a RESTful API Gateway that routes to containerized microservices on managed Kubernetes; images are CDN-served from object storage while transactional data resides in a secured relational database, with a message queue decoupling order intake from fulfillment.

- Web/Mobile Client → API Gateway → Kubernetes-hosted Microservices (Menu, Order, Payment) → RDS (transactional/customer data) + S3/CDN (menu images, invoices).

- Order events are pushed to a message queue; separate consumer services handle kitchen notifications and delivery dispatch asynchronously.
- RBAC and JWT validation occur at the API Gateway before requests reach internal services.
- Autoscaling policies and health checks in Kubernetes ensure the platform remains available during peak demand.

5. Conclusion

The proposed architecture is feasible, scalable, and secure, effectively integrating clients, web APIs, cloud storage, and platform services to satisfy the business requirements outlined in the case, while remaining cost-efficient and maintainable.